ChatGPT

4WD 모바일 로봇 및 TurtleBot 스타일 로봇: 주요 시뮬레이션 파일 요약

일반 4WD 모바일 로봇

파일명	역할	필수 선언 항목
모델링/센서		
robot.urdf.xacro	4WD 로봇의 URDF/Xacro 모델 정의 (섀시, 4개 구동 바퀴, 센서 장착 등). Gazebo 시뮬레이션에서 로봇의 물리적 구조와 센서 위치를 규정.	베이스 링크(예: `base_link`)와 관절(조인트) 정의, 관성 정보 등을 포함. 구동을 위해 Gazebo 차동구동 플러그인(`<plugin>`)을 추가하여 바퀴 조인트를 제어해야 함 [1]. 또한 LiDAR 등의 센서는 링크 및 `<gazebo><sensor>` 태그로 URDF에 포함하고, LaserScan 토픽 (예: `/scan`)과 센서 프레임 이름을 지정하여 시뮬레이터가 데이터를 퍼블리시하도록 설정 [2].
시뮬레이션 환경		
my_world.sdf (또는 사용자 정의 세계 파일)	Gazebo에서 사용할 월드 파일로, 로봇이 동작할 환경(바닥, 장애물 등)을 정의.	물리 엔진 및 환경 객체(예: 지형, 장애물, 벽) 선언이 포함됨. 기본적으로 지면(ground plane)과 중력 등이 설정되어 있어야 하며, 로봇이 스폰될 초기 위치를 지정할 수 있음. (세계 좌표계 원점에서 로봇 위치와 방향 설정)
Launch		
simulation_launch.py (예: Gazebo 실행 런치 파일)	Gazebo 시뮬레이터를 실행하고 URDF 모델을 불러와 로봇을 스폰함. 필요에 따라 RViz 등 시각화 툴도 함께 실행.	`/robot_description` 파라미터에 URDF를 로드하여 Gazebo에 모델을 스폰함 [3]. Gazebo를 빈 월드 또는 지정한 월드와 함께 실행하고, ROS 2의 `spawn_entity` 등을 통해 로봇을 삽입. 모든 노드에 `use_sim_time:=true`를 설정하여 시뮬레이션 시간 사용 [4]. 로봇 상태 발행기(robot state publisher)와 joint 상태 발행기를 시작해 로봇의 TF 및 joint 상태가 퍼블리시되도록 해야 함. (월드 경로, 초기 포즈 등은 런치 파일의 인자로 지정)

파일명	역할	필수 선언 항목
nav2_bringup.launch.py (또는 navigation_launch.py)	Nav2 스택(경로 계획, 제어, 맵 서버/AMCL 등)을 실행하여 자율 주행을 시작하는 런치 파일.	Nav2의 기본 `navigation_launch.py` (nav2_bringup 패키지)을 호출하여 여러 Lifecycle 노드를 올바른 순서로 활성화함 [5]. 이 런치 파일은 `params_file` 인자로 내비게이션 설정 YAML을 받아 각 Nav2 노드에 적용하고, 필요 시 지도 파일 경로(`map` 파라미터)와 `use_sim_time` 플래그를 전달함. 로봇의 좌표계(frame) 설정(예: `map` / `odom` / `base_link`)도 런치 과정에서 Nav2에 전달되며, 센서 데이터 토픽들이 Nav2 노드들과 연결됨.
Nav2 설정		
nav2_params.yaml	Nav2 내비게이션 설정 파일. Global/Local costmap, Planner, Controller 등 Nav2의 모든 모듈 파라미터를 포함.	시뮬레이션에서 필요한 시간/프레임/토픽 설정이 포함되어야 함. 예를 들어 `use_sim_time: true` 설정으로 시뮬레이션 시간을 사용하고, 전역 프레임/로봇 베이스/오돔 프레임(`global_frame: map`, `robot_base_frame: base_link`, `odom_topic: /odom`)을 명시 [6]. 또한 장애물 회피를 위해 레이저 스캔 센서 토픽 이름(`/scan` 등)을 costmap의 입력으로 선언해야 함 [7]. 로봇의 풋프린트 또는 반경, inflation 반경 등 costmap 관련 파라미터와 로컬 플래너/컨트롤러의 속도 제한 등이 정의됨. **주의:** 여기 정의된 프레임 ID들은 URDF의 링크 이름과 일치해야 하며, 일치하지 않을 경우 Nav2가 “Frame ID” 오류를 발생시킴 [8].

TurtleBot 스타일 4WD 로봇

파일명	역할	필수 선언 항목
모델링/센서		

파일명	역할	필수 선언 항목
turtlebot3_<MODEL>.urdf.xacro (예: turtlebot3_waffle_pi.urdf.xacro)	TurtleBot3 로봇의 URDF 모델 파일. 차동구동식 4륜 구조(2륜 구동+캐스터 또는 변형된 4WD)와 LDS 라이다, IMU 등의 센서가 정의되어 있음.	TurtleBot3 모델의 기본 링크 (`base_link`)와 바퀴 조인트, 센서 링크들을 정의. Gazebo 시뮬레이션용 차동구동 플러그인이 URDF에 포함되어 바퀴 회전을 제어하며, 해당 플러그인에서 좌우 바퀴 조인트를 지정해 /cmd_vel 토픽을 받아 구동함 [1]. 또한 상단에 장착된 LDS(LiDAR) 센서는 URDF에 `<gazebo>` 센서 플러그인으로 포함되어 있으며, Gazebo의 `gazebo_ros_ray_sensor` 플러그인을 통해 `/scan` 토픽으로 LaserScan 데이터를 발행하도록 설정됨 (플러그인 내 frame_name으로 LiDAR 링크 지정) [2].
시뮬레이션 환경		
turtlebot3_world.sdf (또는 제공된 다른 환경 월드)	TurtleBot3용 Gazebo 월드 파일. TurtleBot3 예제에 포함된 환경(예: 빈 세계, 집 모양 세계 등)을 정의.	Robotis에서 제공하는 월드에는 기본 지면, 조명 및 일부 장애물 배치가 포함됨. 물리 엔진 파라미터가 기본값으로 설정되어 있고, 로봇이 스폰될 초기 위치도 지정됨. (별도의 환경을 사용할 경우 해당 .world/.sdf 파일을 생성하여 사용 가능)
Launch		
turtlebot3_world.launch.py (Gazebo 실행 런치)	TurtleBot3 로봇을 Gazebo에서 시뮬레이션하기 위한 런치 파일. 선택한 모델(Burger, Waffle, Waffle Pi)에 맞는 URDF를 로드하고 Gazebo 세계를 실행.	이 런치를 실행하기 전에 `TURTLEBOT3_MODEL` 환경 변수를 `burger`/`waffle`/`waffle_pi` 중 해당 로봇으로 설정해야 함 [9]. 런치 파일은 해당 모델의 URDF를 `/robot_description`으로 로드하고, Gazebo에서 지정된 세계를 띄우며, 스폰 스크립트를 통해 로봇을 삽입함. 기본적으로 빈 세계 또는 `turtlebot3_world.sdf`가 로드되고 Gazebo GUI가 표시됨. 모든 노드에 `use_sim_time`가 켜져 시뮬레이션 시간에 동기화됨.

파일명	역할	필수 선언 항목
turtlebot3_navigation.launch.py (Nav2 내비게이션 런치)	TurtleBot3에서 Nav2 자율주행을 시작하는 런치 파일. AMCL을 사용한 자기 위치 추정과 경로 계획 노드들을 실행.	Nav2의 `navigation_launch.py`를 TurtleBot3용으로 래핑하여, TurtleBot3 전용 파라미터를 적용하고 RViz 등을 실행하는 런치 파일. 실행 시 지도 파일(`map` 파라미터) 경로와 모델별 Nav2 파라미터 YAML(예: waffle_pi.yaml)을 인자로 받음. 이 YAML에는 costmap 설정 (장애물 inflation 범위 등)과 로컬 플래너(DWB 등) 튜닝 값이 포함되어 있음 [10] [11] . 런치 파일은 해당 `params/<MODEL>.yaml`을 `params_file`로 지정하여 Nav2 각 모듈에 적용하고, `use_sim_time:=true` 설정을 전달. Nav2 노드들이 사용하는 프레임 ID(`map`, `odom`, `base_link`)와 센서 입력 토픽(`/scan`)도 이 파라미터 파일에 정의되어 있으며, 실제 URDF와 일치하도록 해야 한다.
Nav2 설정		
<MODEL>.yaml (예: waffle_pi.yaml)	TurtleBot3 전용 Nav2 설정 파일(`turtlebot3_navigation2/param/` 디렉토리에 모델별로 제공됨). 해당 모델의 크기와 성능에 맞춰 Nav2 파라미터를 조정.	글로벌/로컬 costmap의 좌표 프레임 설정(`map`, `odom`, `base_link`)과 센서 토픽 (`/scan`) 이름을 명시. 로봇 반경에 맞는 inflation_layer의 `inflation_radius` 등 Costmap 인플레이션 파라미터가 정의되어 있음 [10] . 로컬 플래너(예: DWB)의 최대/최소 속도(`max_vel_x` 등)와 가속도 제한(`acc_lim_*`) 등이 모델에 맞게 설정됨 [12] . 이러한 파라미터들은 TurtleBot3 각 모델 YAML에 사전 설정되어 있으며, Nav2 실행 시 해당 파일을 로드함. (예: Burger와 Waffle은 크기가 다르므로 속도, 코스트맵 크기 등이 다르게 튜닝됨)

[1] Gazebo : Tutorial : Gazebo plugins in ROS
https://classic.gazebosim.org/tutorials?tut=ros_gzplugins

[2] Setting Up Sensors - Gazebo Classic — Nav2 1.0.0 documentation
https://docs.nav2.org/setup_guides/sensors/setup_sensors_gz_classic.html

[3] Using a URDF in Gazebo — ROS 2 Documentation: Humble documentation
https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Using-a-URDF-in-Gazebo.html

[4] [8] GitHub - Ali-Hossam/ROS2_4wheel_autonomous_robot: Four-wheel autonomous robot capable of mapping, localization, and navigation using the ROS 2 framework, Nav2, and AMCL.
https://github.com/Ali-Hossam/ROS2_4wheel_autonomous_robot

[5] Autonomous robot navigation and Nav2: The first steps.
https://foxglove.dev/blog/autonomous-robot-navigation-and-nav2-the-first-steps

[6] [7] ROS 2 Navigation2 Configuration: Complete Guide to Optimizing Your Rob – ThinkRobotics.com
https://thinkrobotics.com/blogs/learn/ros-2-navigation2-configuration-complete-guide-to-optimizing-your-robot-navigation-stack?srsltid=AfmBOooB39gbx2ZEV9YT9dx1da6iYONZNEcS_N_u4WNcKtkxQ6JQIBPV

[9] Error during launching turtlebot3_world.launch - ROS Answers archive
https://answers.ros.org/question/329022/

[10] [11] [12] TurtleBot3
https://emanual.robotis.com/docs/en/platform/turtlebot3/navigation/